1. Expresiones. En la primera entrega las expresiones estaban en niveles (e_or, e_and, e_neg, e_comp, e_aritmetica, termino, potencia, unario, primario). En Rascal se dejó una sola regla Expresion. La precedencia se maneja con > y los atributos left, right y non-assoc, y los paréntesis con bracket. El orden de menor a mayor se mantiene: or, and, neg, comparaciones, + y -, * / %, **, - unario y primarios. La potencia es asociativa a la derecha y las comparaciones no son asociativas, igual que antes.

2. if como expresión. En la primera entrega if_else estaba dentro de primario. En Rascal quedó con la menor prioridad para evitar ambigüedad. Para usarlo dentro de una operación hay que ponerlo entre paréntesis.

3. Partes opcionales y agrupaciones. Para poder usar implode, cada parte necesita su propia regla. Por eso se separaron:

("val" | "var") → Mutabilidad

("+" | "-") en generico → Varianza

(":" tipo)? en las declaraciones → TipoAnotado

("[" tipos "]")? en crear_o → ArgumentosTipo

("(" parametros_constructor? ")")? en objeto → Constructor

4. Listas. Las listas de la forma x ("," x)* se escribieron como {X ","}+ o {X ","}*, que es la notación de Rascal.

5. Espacios en blanco. Además de espacio, tabulación y salto de línea, se acepta \r, porque en Windows los saltos de línea son \r\n.

6. Palabras reservadas. La restricción de que un identifier no sea palabra reservada se implementó con keyword Reserved y restando Reserved en Identifier. Se mantuvo el guion como carácter válido en los identificadores. Por eso a-b se interpreta como un solo identificador y la resta debe escribirse con espacios (a - b).

7. Nombres. Se renombraron algunas reglas para mayor claridad:

a_miembro → AccesoMiembro

a_metodo → LlamadaMetodo

crear_o → CrearObjeto

tipo_b, tipo_o, tipo_g → tipoBasico, tipoObjeto, tipoGenerico

unario → negativo